Noondeterministic and Probabilistic Algorithms

March 4, 2026

1 Nondeterministic Algorithms

We have seen so far some deterministic algorithms. An algorithm is deterministic if it has a single possible execution path. Every computation step follows a predefined sequence of operations, ensuring that the same input always produces the same output.

> An algorithm is **deterministic** if each instruction it contains has a uniquely determined result.

Another model of computation allows for nondeterminism, that is, the execution path is not uniquely determined. Instead, there may be more ways to continue the execution. For the same input data, depending on the chosen execution path, there may be multiple (different) results.

A nondeterministic algorithm can explore multiple computational paths simultaneously. In the **angelic version**, the algorithm "guesses" the execution that leads to the correct result. The execution time is the time of the execution that leads to the correct result.

> An algorithm is **nondeterministic** if it contains instructions whose result is not uniquely determined, but a value from a finite set of possible values.

An essential feature of a nondeterministic algorithm is its unrestricted parallelism: each possible execution path runs simultaneously (with each individual path being run deterministically). Nondeterministic algorithms use the following operations:

- **choose x from A**, where A is a finite set: at runtime, a copy of the algorithm is made for every value of A, with all copies being run independently and simultaneously.
- **success**: successfully ends the execution of the current execution path (a solution was found), and of the entire algorithm (all other execution paths).
- **failure**: terminates with failure the execution of the current execution path. The other paths are not affected.

If there exists a successful execution for some input data, the nondeterministic algorithm will execute it and, by use of **success**, end this and all other execution paths. So you can imagine it simply guesses the correct choice that leads to the result.

Generally, a nondeterministic algorithm has two phases:

- **Guessing phase:** the algorithm nondeterministically chooses (guesses) a solution.

```
choose x from {2,3,4}
if x MOD 2 =0 then
    success
else
    failure
```

- **Verification phase:** the algorithm checks the correctness of the guessed solution.

Consider the following nondeterministic algorithm:

The algorithm checks if there exists an even number among {2, 3, 4}. The algorithm has two successful choices, x=2 and x=4, so it will end in success.

Consider another nondeterministic algorithm:

```
choose x from {1,2,3}
choose y from {5,6}
if (x+y) MOD 5 =0 then
    success
else
    failure
```

The algorithm checks whether, choosing a value among {1, 2, 3} and one from {5, 6}, the sum of the two values is a multiple of 5. The algorithm has 3*2=6 execution paths, none of which is successful, so it will end in failure.

For the Boolean Satisfiability (SAT) problem, a nondeterministic algorithm would guess an assignment, then check whether the given formula is true in that assignment. If yes, it would end in success, otherwise in failure.

We say that a nondeterministic algorithm A solves a problem P if for any instance x of problem P, there exists a non-failing execution that is terminating, and whose final configuration includes $P(x)$.

Theorem: For any non-deterministic algorithm A there is an equivalent deterministic algorithm B, which has the worst case execution time $T_B(n) = O(2^{T_A(n)})$.

In Table 1 we present a comparison between deterministic and nondeterministic algorithms.

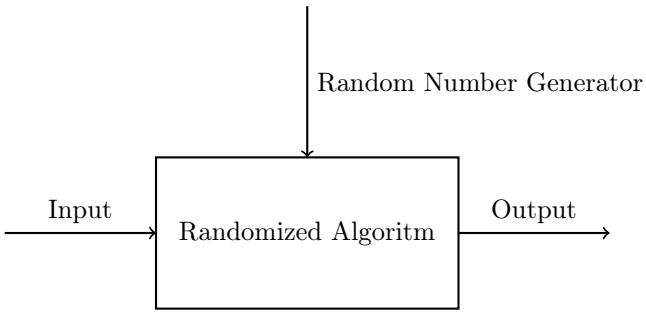
Feature	**Deterministic Alg.**	**Nondeterministic Alg.**
Execution Path	Single, well-defined	Multiple possible paths
Output for Same Input	Always the same	May vary based on choices
Use of Randomness	None	Can involve "guessing" or randomization

Table 1: Comparison of deterministic and nondeterministic Algorithms

2 Probabilistic Algorithms

Probabilistic (randomized) algorithms are a powerful and widely used class of algorithms that incorporate **randomness** into their decision-making process to improve efficiency, simplicity, or robustness. Unlike deterministic algorithms, for which the input completely determines the

sequence of computations performed by the algorithm, randomized algorithms make random choices at certain points during execution, leading to different possible outcomes for the same input. In addition to its input data, a randomized algorithm has access to a stream of random bits that it can use for the purpose of making random choices. Even when the input is fixed, the execution time of a randomized algorithm is a random variable.



There are two main advantages that randomized algorithms often have. First, often the **execution time** or space requirement of a randomized algorithm is smaller than that of the best deterministic algorithm that we know for the same problem. For example, the Agrawal–Kayal–Saxena (AKS) primality test was the first deterministic polynomial time algorithm, with an initial complexity of $O(log^{12}n)$, and had a huge theoretical significance. However, it is not used in practice because randomized algorithms like Miller-Rabin are much faster.

Second, randomized algorithms are usually **extremely simple** to understand and to implement; often, the introduction of randomization suffices to convert a simple and naive deterministic algorithm with bad worst-case behavior into a randomized algorithm that performs well with high probability on every possible input.

These algorithms are particularly useful for solving problems where deterministic approaches may be inefficient or complex. Randomized algorithms can be broadly classified into **Las Vegas algorithms**, which always produce correct results but have variable runtimes, and **Monte Carlo algorithms**, which run in bounded time but may yield incorrect answers with some probability (that can be decreased to a desired margin at the expense of execution time).

Consider the following problem: Given an array a of $n \geq 2$ elements with equal number of 0s and 1s, output an index of a 1. Below, you have two algorithms for the problem.

Algorithm 1 Infinite Random Search

Require: Array $a[0 \dots n-1]$
1: **while** true **do**
2: $k \leftarrow \text{Random}(0, n-1)$
3: **if** $a[k] = 1$ **then**
4: **return** k

Algorithm 2 α-Bounded Random Search

Require: Array $a[0 \dots n-1]$, integer α
1: **for** $i = 1$ to α **do**
2: $k \leftarrow \text{Random}(0, n-1)$
3: **if** $a[k] = 1$ **then**
4: **return** k
5: **return** `"failed"`

The algorithm on the left doesn't gamble with correctness, but it gambles with run time. It may happen that the random choice at line 2 is an index of a 0 for a long time (although, with decreasing probability). This is an example of a Las Vegas algorithm.

The algorithm on the right doesn't gamble with run time (it makes at most α checks), but it gambles with correctness (if all random values of k at line 2 were indices of 0, it will return

"failed", although we know the input contains 1s). This is a example of a Monte Carlo algorithm. The probability of failure is $\frac{1}{2^\alpha}$.

2.1 Random variables

A (discrete) random variable X is a function from a finite or countably infinite sample space S to the real numbers. It associates a real number with each possible outcome of an experiment, which allows us to work with the probability distribution induced on the resulting set of numbers.

For a random variable X and a real number x, we define the event $X = x$ to be $\{s \in S | X(s) = x\}$, thus

$\Pr\{X = x\} = \sum_{s \in S | X(s)=x} \Pr\{s\}$

The function $f(x) = \Pr\{X = x\}$ is the **probability density function** of the random variable X. From the probability axioms, $\Pr\{X = x\} \geq 0$ and $\sum_x \Pr\{X = x\} = 1$.

Examples:

1. D (rolling a die)

   - random phenomenon: rolling a die
   - D returns the number on the die

i	1	2	3	4	5	6
x_i	1	2	3	4	5	6
p_i	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$	$\frac{1}{6}$

Table 2: Probability distribution of a fair die

C++ coode:

```
#include <random>
int D() {
   static std::random_device rd;
   static std::mt19937 gen(rd());
   static std::uniform_int_distribution<int> dist(1, 6);
   return dist(gen);
}
```

2. SD2 (sum of two dice)

   - random phenomenon: rolling two dice
   - SD2 returns the sum of the two numbers on the dice

i	1	2	3	4	5	6	7	8	9	10	11
x_i	2	3	4	5	6	7	8	9	10	11	12
p_i	$\frac{1}{36}$	$\frac{2}{36}$	$\frac{3}{36}$	$\frac{4}{36}$	$\frac{5}{36}$	$\frac{6}{36}$	$\frac{5}{36}$	$\frac{4}{36}$	$\frac{3}{36}$	$\frac{2}{36}$	$\frac{1}{36}$

Table 3: Probability distribution of the sum of two fair dice

We have a total of 6*6=36 possible dice rolls. Pr{X=2} = $\frac{1}{36}$, as a sum of 2 can only be obtained by having 1 on the top faces of both dice. Pr{X=4} = $\frac{3}{36}$, as a sum of 4 can be obtained by having (1,3), (2,2) or (3,1) on the top faces of the dice.

3. CB (chocolate bar)
   - random phenomenon: randomly select uniformly at random where to break a chocolate bar of n equal pieces (breaking between the pieces) and select the larger piece of chocolate.
   - CB returns $max\left(\frac{i}{n}, \frac{n-i}{n}\right), i = 1, 2, ..., n-1$

Expected value of a random variable

The simplest and most useful summary of the distribution of a random variable is the "average" of the values it takes on. The **expected value** (or, synonymously, **expectation** or **mean**) of a discrete random variable X is

$$E[X] = \sum_x x \cdot \Pr\{X = x\}$$

Properties

$$E[X + Y] = E[X] + E[Y]$$

$$E[X] = E[X] \cdot E[Y], \text{where X and Y are independent}$$

When a random variable X takes on values from the set of natural numbers $\mathbb{N}$, we have

$$E[X] = \sum_{i=1}^{\infty} \Pr\{X \geq i\}$$

Expected value of CB

1. If n is odd:
   - possible values for CB: $\left\{\frac{k}{n} \middle| k = n-1, n-2, ..., \frac{n+1}{2}\right\}$, each with probability $\frac{2}{n-1}$.
   - $E[CB] = \sum_{k=\frac{n+1}{2}}^{n-1} \frac{k}{n} \cdot \frac{2}{n-1} = \frac{2}{n(n-1)}\left(\frac{n(n-1)}{2} - \frac{n^2-1}{8}\right) = \frac{3n^2-4n+1}{4n(n-1)}$

     $\lim_{n\to\infty} E[CB] = \lim_{n\to\infty} \frac{3n^2-4n+1}{4n(n-1)} = \lim_{n\to\infty} \frac{3-\frac{4}{n}+\frac{1}{n^2}}{4\left(1-\frac{1}{n}\right)} = \frac{3}{4}$.
2. If n is even : try it yourself. You should get the same result.

2.2 Las Vegas algorithms

Las Vegas algorithms are a class of randomized algorithms that always produce a **correct solution**, but whose running time is **variable** depending on the random choices used during execution. Unlike Monte Carlo algorithms, which may return incorrect answers with some probability, Las Vegas algorithms guarantee correctness, with the tradeoff of execution time. The random choices made within the algorithm are used to establish an expected running time for the algorithm that is, essentially, independent of the input.

These algorithms are particularly useful when exact solutions are required, but deterministic approaches are too slow or complex. A common example is Randomized Quicksort, where the

pivot is chosen randomly, ensuring an average-case time complexity of $O(n\log n)$, though the exact execution time may vary. Las Vegas algorithms are widely used in computational geometry, sorting, and optimization problems where correctness is non-negotiable but efficiency can be improved with randomness.

QuickSort

QuickSort, like merge sort, applies the divide-and-conquer paradigm. The steps for sorting a typical subarray $A[p..r]$ are the following:

Divide: Partition (rearrange) the array $A[p..r]$ into two (possibly empty) subarrays $A[p..q-1]$ and $A[q+1..r]$ such that each element of $A[p..q-1]$ is less than or equal to $A[q]$, which is, in turn, less than or equal to each element of $A[q+1..r]$. Computing the index q is part of this partitioning procedure.

Conquer: Sort the two subarrays $A[p..q-1]$ and $A[q+1..r]$ by recursive calls to QuickSort.

Combine: Because the subarrays are already sorted, no work is needed to combine them. The entire array $A[p..r]$ is now sorted.

Algorithm 3 QuickSort

```
function QUICKSORT(A, p, r)
    if p < r then
        q ← PARTITION(A, p, r)
        QUICKSORT(A, p, q − 1)
        QUICKSORT(A, q + 1, r)
```

To sort an entire array A, the initial call is QUICKSORT$(A, 1, A.length())$.

Algorithm 4 Partitioning the array

```
function PARTITION(A, p, r)
    x ← A[r]                                  ▷ Pivot element
    i ← p − 1
    for j ← p to r − 1 do
        if A[j] ≤ x then
            i ← i + 1
            Swap A[i] and A[j]
    Swap A[i + 1] and A[r]
    return i + 1
```

The PARTITION function always uses the rightmost element as the pivot for partitioning. You can see a proof of correctness in [1], Chapter 7.

Worst-case partitioning

The worst-case behavior for QuickSort occurs when the partitioning routine produces one subproblem with $n-1$ elements and one with 0 elements. Assuming this unbalanced partitioning arises in each recursive call, we get the recurrence $T(n) = T(n-1) + \Theta(n)$, which intuitively gives an arithmetic series, which evaluates to $\Theta(n^2)$.

Best-case partitioning

In the most even possible split, PARTITION produces two subproblems, each of size no more than $n/2$. In this case, QuickSort runs much faster. The recurrence for the running time is then $T(n) = 2T(n/2) + \Theta(n)$, which we saw last time evaluates to $\Theta(n\log n)$.

By equally balancing the two sides of the partition at every level of the recursion, we get an asymptotically faster algorithm.

A randomized version of QuickSort

Instead of always using the rightmost element as the pivot, we will randomly choose an element from the subarray $A[p..r]$. We do so by first exchanging $A[r]$ with a randomly selected element from $A[p..r]$. By randomly sampling the range $p..r$, we ensure that the pivot element $x = A[r]$ is equally likely to be any of the $r-p+1$ elements in the subarray. Because we randomly choose the pivot element, we expect the split of the input array to be reasonably well balanced on average. The changes to QUICKSORT and PARTITION are minimal. We only introduce the swap of $A[r]$ with a random element before partitioning.

Algorithm 5 Randomized Partition

```
function RANDOMIZED-PARTITION(A, p, r)
    i ← RANDOM(p, r)
    Swap A[r] and A[i]
    return PARTITION(A, p, r)
```

Algorithm 6 Randomized QuickSort

```
function RANDOMIZED-QUICKSORT(A, p, r)
    if p < r then
        q ← RANDOMIZED-PARTITION(A, p, r)
        RANDOMIZED-QUICKSORT(A, p, q − 1)
        RANDOMIZED-QUICKSORT(A, q + 1, r)
```

Time analysis of RANDOMIZED-QUICKSORT

The running time of RANDOMIZED-QUICKSORT is dominated by the time spent in the PARTITION procedure. Each time the PARTITION procedure is called, it selects a pivot element, and this element is never included in any future recursive calls to RANDOMIZED-QUICKSORT and PARTITION. Thus, there can be at most n calls to PARTITION over the entire execution of the quicksort algorithm. One call to PARTITION takes $O(1)$ time plus an amount of time that is proportional to the number of iterations of the for loop in lines 4–7. Each iteration of this for loop performs a comparison in line 5, comparing the pivot element to another element of the array A. Therefore, if we can count the total number of times that line 5 is executed, we can bound the total time spent in the for loop during the entire execution of RANDOMIZED-QUICKSORT.

Define the random indicator variable

$$X_{ij} = \begin{cases} 1 & \text{if the } i\text{-th and } j\text{-th element in the sorted list are compared} \\ 0 & \text{otherwise.} \end{cases}$$

Since each pair is compared at most once, we can easily characterize the total number of comparisons performed by the algorithm. Let $C(n)$ be the random variable that returns the number of comparisons:

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$$

The expected number of comparisons is:

$$\begin{aligned} E[C(n)] &= E\left[\sum_{i=1}^{n-1}\sum_{j=i+1}^{n} X_{ij}\right] \\ &= \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} E[X_{ij}] \end{aligned}$$

Let p_{ij} be the probability that the i-th and the j-th elements are compared. The i-th and the j-th elements (in the sorted list) are compared iff the first pivot from $a_i..a_j$ is a_i or a_j. It follows that $p_{ij} = \frac{2}{j+1-i}$.

We get

$$\begin{aligned} E[C(n)] &= \sum_{i=1}^{n-1}\sum_{j=1+1}^{n} p_{ij} \\ &= \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} \frac{2}{j+1-i} \\ &= \sum_{i=1}^{n-1}\sum_{k=1}^{n-i} \frac{2}{k+1} \\ &< \sum_{i=1}^{n-1}\sum_{k=1}^{n} \frac{2}{k} \\ &= 2\sum_{i=1}^{n-1} H_n \end{aligned}$$

The harmonic series H_n grows similarly to $\log n$, so we get an $O(\log n)$ bound from it, we can ignore the constant 2 to obtain $\sum_{i=1}^{n-1} O(\log n)$ which yields an expected running time $O(n\log n)$.

2.3 Monte Carlo algorithms

Monte Carlo algorithms are a class of randomized algorithms that trade deterministic correctness for efficiency. They provide a probabilistic guarantee of correctness, meaning they may produce incorrect results with a small probability. These algorithms are particularly useful when exact solutions are computationally expensive, but an approximate or probabilistic solution is acceptable.

Monte Carlo algorithms typically have bounded running time but introduce randomness in their decision-making process. They are widely used in numerical simulations, optimization, and approximation algorithms.

Primality testing

Let us consider the primality problem from the first lecture: given a number, is it prime or not?

PRIMES
INPUT: A number $k \in \mathbb{N}$;
OUTPUT: $\begin{cases} \text{Yes, if } k \text{ MOD } i \neq 0, \quad \forall i \in \mathbb{N}, \quad 1 < i \leq \sqrt{k}, \\ \text{No, if } \exists a, b \in \mathbb{N},\ a, b \neq 1 \text{ s.t. } k = a * b. \end{cases}$

A naive algorithm for solving the problem would look something like Algorithm 7. We will introduce some mathematical background and then the Solovay-Strassen randomized algorithm for solving the same problem.

Algorithm 7 Naive Primality Test

```
function IsPrime(k)
    if k ≤ 1 then return false
    for i ← 2 to √k do
        if k mod i = 0 then return false
    return true
```

Legendre symbol

The **Legendre symbol** is a function that determines whether a number is a quadratic residue modulo a prime. It is defined for an odd prime p and an integer a, denoted as: $\left(\frac{a}{p}\right)$

$$\left(\frac{a}{p}\right) = \begin{cases} 1, & \text{if } a \text{ is a quadratic residue modulo } p \text{ and } a \not\equiv 0 \mod p \\ -1, & \text{if } a \text{ is a non-quadratic residue modulo } p \\ 0, & \text{if } a \equiv 0 \mod p \end{cases}$$

This function determines whether there exists an integer x such that:

$$x^2 \equiv a \pmod p$$

Properties of the Legendre Symbol:

1. Congruence Property:

$$\text{If } a \equiv b \pmod n, \text{ then } \left(\frac{a}{n}\right) = \left(\frac{b}{n}\right).$$

2. Multiplicative Property:

$$\left(\frac{a \cdot b}{p}\right) = \left(\frac{a}{p}\right) \cdot \left(\frac{b}{p}\right)$$

3. Euler's Criterion:

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod p$$

This formula allows efficient computation of the Legendre symbol.

4. Gauss's Law off Quadratic Reciprocity: Suppose a and p are distinct odd primes. Then

$$\left(\frac{a}{p}\right) = (-1)^{\frac{(a-1)(p-1)}{4}} \left(\frac{p}{a}\right)$$

Also $\left(\frac{-1}{p}\right) = (-1)^{\frac{p-1}{2}}$ and $\left(\frac{2}{p}\right) = (-1)^{\frac{p^2-1}{8}} = \begin{cases} 1 & \text{if } p \equiv \pm 1 \pmod 8, \\ -1 & \text{if } p \equiv \pm 3 \pmod 8. \end{cases}$

Obs: There are always $\frac{p-1}{2}$ quadratic residues modulo a prime p. If a is a quadratic residue, then its negative modulo p is a quadratic non-residue.

You can find more properties, examples and proofs in Chapter 4 of [2].

Jacobi symbol

The **Jacobi symbol** is a generalization of the Legendre symbol, extending its definition to any odd composite number. It is denoted as: $\left(\frac{a}{n}\right)$ where n is an **odd positive integer** with prime factorization:

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$$

The Jacobi symbol is defined as:

$$\left(\frac{a}{n}\right) = \left(\frac{a}{p_1}\right)^{e_1} \times \left(\frac{a}{p_2}\right)^{e_2} \times \cdots \times \left(\frac{a}{p_k}\right)^{e_k}$$

where each $\left(\frac{a}{p_i}\right)$ is a Legendre symbol.

We have the following:

- If n is prime, the Jacobi symbol equals the Legendre symbol.
- If n is composite, the Jacobi symbol does not directly indicate whether a is a quadratic residue modulo n.
- The Jacobi symbol can be computed efficiently without factoring n.

Properties of the Jacobi symbol. Let n be an odd positive integer, and let $a, b \in \mathbb{Z}$ be coprime to n. The Jacobi symbol possesses the following properties:

1. Congruence Property:

$$\text{If } a \equiv b \pmod n, \text{ then } \left(\frac{a}{n}\right) = \left(\frac{b}{n}\right).$$

2. Multiplicative Property (m is an odd positive integer coprime to n):

$$\left(\frac{a \cdot b}{n}\right) = \left(\frac{a}{n}\right) \cdot \left(\frac{b}{n}\right)$$

3. Law of Quadratic Reciprocity:

$$\left(\frac{m}{n}\right) = (-1)^{\frac{(m-1)(n-1)}{4}} \left(\frac{n}{m}\right)$$

$$\text{Also, } \left(\frac{-1}{n}\right) = (-1)^{\frac{n-1}{2}} = \begin{cases} 1 & \text{if } n \equiv 1 \pmod 4, \\ -1 & \text{if } n \equiv 3 \pmod 4 \end{cases}$$

$$\text{and } \left(\frac{2}{n}\right) = (-1)^{\frac{n^2-1}{8}} = \begin{cases} 1 & \text{if } n \equiv \pm 1 \pmod 8, \\ -1 & \text{if } n \equiv \pm 3 \pmod 8. \end{cases}$$

We get

$$\left(\frac{m}{n}\right) = \begin{cases} \left(\frac{n}{m}\right) & \text{if } m \equiv 1 \pmod 4 \text{ or } n \equiv 1 \pmod 4, \\ -\left(\frac{n}{m}\right) & \text{if } m \equiv 3 \pmod 4 \text{ and } n \equiv 3 \pmod 4. \end{cases}$$

This is useful for computing the symbol efficiently.

Note that Euler's criterion was not listed as a property of the Jacobi symbol. This is because it may happen for some $1 \leq a \leq n-1$ that $\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod n$, even though n is not prime.

Algorithm 8 gives the pseudocode for computing the Jacobi symbol.

Algorithm 8 Compute the Jacobi Symbol $\left(\frac{a}{n}\right)$

```
function JacobiSymbol(a, n)
    if n ≤ 0 or n is even then return undefined
    a ← a mod n                                   ▷ Reduce a modulo n
    j ← 1
    while a ≠ 0 do
        while a is even do
            a ← a/2
            if n mod 8 = 3 or n mod 8 = 5 then
                j ← −j
        swap a, n
        if a ≡ 3 (mod 4) and n ≡ 3 (mod 4) then
            j ← −j
        a ← a mod n
    if n = 1 then return j
    else
        return 0
```

The Solovay-Strassen algorithm

We have that if n is prime, then the Jacobi and Legendre symbols agree, and we have $\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod n$. If n is composite, for some $1 \leq a \leq n-1$ it may hold that $\left(\frac{a}{n}\right) \equiv a^{(n-1)/2} \pmod n$, but for some we will get $\left(\frac{a}{n}\right) \not\equiv a^{(n-1)/2} \pmod n$. We call a number a for which either $(a, n) > 1$ or $(a, n) = 1$ and $\left(\frac{a}{n}\right) \not\equiv a^{(n-1)/2} \pmod n$ a witness for the compositeness of n. For an odd composite number n, more than half of the numbers $1 \leq a \leq n-1$ are witnesses for it being composite. The Solovay-Strassen algorithm tries to find such witnesses, with randomly chosen values of a. If a is a witness, then n is definitely composite, otherwise it may be prime.

Let us consider an efficient way of computing the power function modulo, based on the following definition:

Algorithm 9 Solovay-Strassen Primality Test

```
function SolovayStrassen(n)
    if n ≤ 1 or n is even > 2 then return Composite
    Choose a random integer a s.t. 2 ≤ a ≤ n − 1
    if gcd(a, n) ≠ 1 then return Composite
    j ← JacobiSymbol(a, n)                ▷ Compute the Jacobi symbol J = (a/n)
    Compute x = a^((n−1)/2) (mod n)
    if j == 0 or x ≢ j (mod n) then return Composite
    return ProbablyPrime
```

$$a^n \pmod p = \begin{cases} 1 & \text{if } n = 0, \\ a \pmod p & \text{if } n = 1, \\ (a^{\frac{n}{2}} \pmod p) * (a^{\frac{n}{2}} \pmod p) \pmod p & \text{if } n\%2 == 0, \\ (a * a^{n-1}) \pmod p & \text{if } n\%2 == 1. \end{cases}$$

Algorithm 10 computes the power modulo function in $O(\log n)$, making it efficient for large exponents.

Algorithm 10 Modular Exponentiation (Power Modulo)

```
function PowerMod(a, n, p)
    r ← 1
    while n > 0 do
        if n%2 == 1 then
            r ← r * a (mod p)
        a ← a * a (mod p)
        n ← n/2
    return r
```

We can modify Algorithm 9 to give an answer with a certain probability, by passing it an extra argument k as a count for the number of times we test for compositeness with a new a.

Algorithm 11 Solovay-Strassen Primality Test

```
function SolovayStrassen(n, k)
    if n ≤ 1 or n is even >2 then return Composite
    for i = 1 to k do
        Choose a random integer a s.t. 2 ≤ a ≤ n − 1
        if gcd(a, n) ≠ 1 then return Composite
        j ← JacobiSymbol(a, n)
        x ← PowerMod(a, (n − 1)/2, n)
        if j == 0 or x ≢ j (mod n) then return Composite
    return ProbablyPrime
```

We mentioned that if n is composite, then more than half of the values $< n$ are witnesses to its compositeness, so the algorithm is likely to find one. The probability with which Algorithm 11 returns "probably prime" for a composite number (false positive) is $1/2^k$.

A Mathematical background, terminology and notations

This section is a reminder of several theoretical concepts related to counting theory and probability theory that are useful both for following the lecture and solving seminar exercises. Feel free to skip this section if you already understand these concepts. If you need even more details and examples, read Appendix C of [1], on which this section is based.

A.1 Counting theory

Counting theory aims to answer questions like "How many n-bit numbers are there?" without actually enumerating all the choices.

A.1.1 Rules of sum and product

The **rule of sum** says that if we want to choose an element from one of two disjoint sets, the number of ways in which we can do this is the **sum** of the cardinalities of the two sets.

More formally, given A and B two disjoint sets (they share no common element), we have |A ∪ B| = |A| + |B|, where |A| is the cardinality (number of elements) of set A.

For example, assuming that a username can only contain lowercase letters, digits or the special characters underscore (_) or dot (.), a character in the username can be one of $26 + 10 + 2 = 38$ possibilities (26 letters, 10 digits, or one of the two special characters).

If we are interested in selecting ordered pairs from two finite sets of elements, A and B, we will use the **rule of product**: the number of ways to choose an ordered pair is a product of the number of ways to choose the first element (|A|) and the number of ways to choose the second element (|B|), that is |A×B| = |A|×|B|.

For example, if a group consists of 18 boys and 7 girls, there are 18×7 ways of forming boy-girl dance pairs for a ball.

A.1.2 Strings

A **string** over a finite set S is a sequence of elements from the set S. For example, there are 8 binary strings of length 3: 000, 001, 010, 011, 100, 101, 110, 111.

A string of length k is sometimes called a ***k-string***. A **substring** *s'* of a string s is an ordered sequence of consecutive elements of s. For example, the empty string, 0, 1, and 01 are all substrings of 01. A ***k-substring*** of a string s is a substring of s that has length k. For example, 001 and 010 are 3-substrings of 0001010, but 011 is not a 3-substring of the same string.

A k-string over a set S can be seen as an element of the Cartesian product of k instances of set S, thus using the product rule we know there are $|S|^k$ strings of length k. The number of binary strings of length k is 2^k.

A.1.3 Permutations

A **permutation** of a finite set S is an ordered sequence of all the elements of S, each element appearing exactly once. For example, the set S=$\{0, 1, 2\}$ has 6 permutations: $\{0, 1, 2\}$, $\{0, 2, 1\}$, $\{1, 0, 2\}$, $\{1, 2, 0\}$, $\{2, 0, 1\}$, $\{2, 1, 0\}$.

For a set of n elements there are $n!$ permutations possible. There are n possible ways of choosing the first element. With the first element fixed, there remain $n - 1$ ways of choosing the second element in the permutation. With two set elements, there are $n - 2$ possibilities left to choose the third element, and so on until there will be only one option of choosing the last element in a permutation. This totals to $n(n - 1)(n - 2) \ldots 1 = n!$.

A$k-$**permutation** of a set S is a permutation of k elements of S, each element appearing at most once in the permutation. For example, the 2-permutations of the set S=$\{0, 1, 2\}$ are: $\{0, 1\}$, $\{0, 2\}$, $\{1, 0\}$, $\{1, 2\}$, $\{2, 0\}$, $\{2, 1\}$.

The number of k-permutations for a set of size n is $n(n-1)(n-2)\ldots(n-k+1) = \dfrac{n!}{(n-k)!}$, since we have n possibilities of choosing the first element, $n-1$ possibilities of choosing the second element and so on, until we get to choose the k-th element from the remaining $n-k+1$ elements.

A.1.4 Combinations

A k**-combination** of an n-set S is a k-subset of S (the order of the elements is not relevant, what matters is whether they are in the subset or not). The 2-combinations of set S=$\{0, 1, 2\}$ are $\{0, 1\}$, $\{0, 2\}$, and $\{1, 2\}$.

We can think of a k-combination as a k-permutation where the order of the elements is irrelevant. There are $k!$ permutations of a k-combination (it has k elements). Given that there are $\dfrac{n!}{(n-k)!}$ possible k-permutations for an n-set S, dividing this by the number of possible permutations of each subset of k elements gives us the total number of combinations, $\dfrac{n!}{k!(n-k)!}$.

A.1.5 Binomial coefficients

We use the notation $\binom{n}{k}$ (read n choose k) for the number of k-combinations of an n-set.

$\binom{n}{k} = \dfrac{n!}{k!(n-k)!}$.

The formula is symmetric in k and $n-k$: $\binom{n}{k} = \dfrac{n!}{k!(n-k)!} = \dfrac{n!}{(n-k)!k!} = \binom{n}{n-k}$.

These numbers are also known as **binomial coefficients** because they appear in the binomial expansion:

$$(x+y)^n = \sum_{k=0}^{n} \binom{n}{k} x^k y^{n-k}. \tag{1}$$

A special case of the binomial expansion is when $x = y = 1$. The expansion becomes:

$$2^n = \sum_{k=0}^{n} \binom{n}{k}. \tag{2}$$

This formula corresponds to counting the 2^n binary n-strings by the number of 1s they contain: $\binom{n}{k}$ binary n-strings contain exactly k 1s, since there are $\binom{n}{k}$ ways of choosing the k out of n positions where to place a 1.

A.2 Probabilities

We define probability in terms of a **sample space** S, a set whose elements are called **elementary events**. An elementary event can be thought of as a possible result of an experiment. For example, if the experiment is flipping two coins, the possible outcomes (and thus, the sample

space) are: HH, HT, TH, TT, where H denotes head, and T denotes tail, and the position in the string indicates the first, respectively the second coin.

An **event** is a subset of the sample space S. For example, the event of obtaining one head and one tail is HT, TH. The event $\varnothing$ is calledd the **null event**, and S is called the **certain event**. Two events A and B are said too be **mutually exclusive** if $A \cap B = \varnothing$. By definition, all elementary events are mutually exclusive.

A.2.1 Axioms of probability

A **probability distribution** Pr{} on a sample space S is a mapping from events of S to real numbers satisfying the following probability axioms:

1. $\Pr\{A\} \geq 0$, for any event A.
2. $\Pr\{S\} = 1$.
3. $\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\}$ for any mutually exclusive events A and B. More generally, for any sequence of pairwise mutually exclusive events $A_1, A_2, \ldots$: $\Pr\left\{\bigcup_i A_i\right\} = \sum_i \Pr\{A_i\}$.

$\Pr\{A\}$ is the **probability** of event A. There are several results that immediately follow from the three axioms above and basic set theory:

- The null event $\varnothing$ has probability $\Pr\{\varnothing\} = 0$.
- If $A \subseteq B$, then $\Pr\{A\} \leq \Pr\{B\}$.
- Let $\overline{A}$ denote the complement of A (S$-A$). Then $\Pr\{\overline{A}\} = 1 - \Pr\{A\}$.
- For any two events A and B, $\Pr\{A \cup B\} = \Pr\{A\} + \Pr\{B\} - \Pr\{A \cap B\} \leq \Pr\{A\} + \Pr\{B\}$.

In the example of flipping two coins, let's assume each elementary event is equiprobable (thus, probability 1/4). The probability of not obtaining two heads is $\Pr\{\overline{\text{HH}}\} = 1 - \Pr\{\text{HH}\} = 1 - 1/4 = 3/4$. The probability of obtaining at least one tail is $\Pr\{\text{HT, TH, TT}\} = \Pr\{\text{HT}\} + \Pr\{\text{TH}\} + \Pr\{\text{TT}\} = 1/4 + 1/4 + 1/4 = 3/4$.

A.2.2 Discrete probability distributions

A probability distribution is **discrete** if it is defined over a finite or countably infinite sample space. Let S be the sample space. Then for any event A, $\Pr\{A\} = \sum_{x \in A} \Pr\{x\}$, since the elementary events in A are mutually exclusive.

If S is finite and each elementary element is equiprobable $\left(\Pr\{s\} = \frac{1}{|S|}\right)$, then we have the **uniform probability distribution** on S. The event in this case is described as ***picking an element of S at random***.

Conditional probability and independence

Conditional probability formalizes the notion of having prior partial knowledge of the outcome of an experiment. The conditional probability of an event A given that another event B occurs is defined to be

$$\Pr\{A|B\} = \frac{\Pr\{A \cap B\}}{\Pr\{B\}}$$

Two events are **independent** if $\Pr\{A \cap B\} = \Pr\{A\}\Pr\{B\}$.

A.3 Discrete random variables

A (discrete) random variable X is a function from a finite or countably infinite sample space S to the real numbers. It associates a real number with each possible outcome of an experiment, which allows us to work with the probability distribution induced on the resulting set of numbers.

For a random variable X and a real number x, we define the event $X = x$ to be $\{s \in S | X(s) = x\}$, thus

$\Pr\{X = x\} = \sum_{s \in S | X(s)=x} \Pr\{s\}$

The function $f(x) = \Pr\{X = x\}$ is the **probability density function** of the random variable X. From the probability axioms, $\Pr\{X = x\} \geq 0$ and $\sum_x \Pr\{X = x\} = 1$.

Expected value of a random variable

The simplest and most useful summary of the distribution of a random variable is the "average" of the values it takes on. The **expected value** (or, synonymously, expectation or mean) of a discrete random variable X is

$$E[X] = \sum_x x \cdot \Pr\{X = x\}$$

Properties

$$E[X + Y] = E[X] + E[Y]$$

$$E[XY] = E[X] \cdot E[Y], \text{where X and Y are independent}$$

When a random variable X takes on values from the set of natural numbers $\mathbb{N}$, we have

$$E[X] = \sum_{i=1}^{\infty} \Pr\{X \geq i\}$$

A.4 Indicator random variables

In order to analyze many algorithms we use indicator random variables. Indicator random variables provide a convenient method for converting between probabilities and expectations. Suppose we are given a sample space S and an event A. Then the indicator random variable $I\{A\}$ associated with event A is defined as

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs} \\ 0 & \text{if } A \text{ does not occur} \end{cases}$$

As an example, let us determine the expected number of heads that we obtain when flipping a fair coin. We can then define an indicator random variable X_H , associated with the coin coming up heads, which is the event H. This variable counts the number of heads obtained in this flip, and it is 1 if the coin comes up heads and 0 otherwise. We write

$$X_H = I\{H\} = \begin{cases} 1 & \text{if } H \text{ occurs} \\ 0 & \text{if } T \text{ occurs} \end{cases}$$

The expected number of heads obtained in one flip of the coin is simply the expected value of our indicator variable X_H :

$$\begin{aligned} E[X_H] &= E[I\{H\}] \\ &= 1 \cdot Pr\{H\} + 0 \cdot Pr\{T\} \\ &= 1/2 \end{aligned}$$

Lemma Given a sample space S and an event A in the sample space S, let $X_A = I\{A\}$. Then $E[X_A] =$Pr$\{A\}$.

References

[1] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 2009.

[2] William Stein. *Elementary number theory: primes, congruences, and secrets: a computational approach*. Springer Science & Business Media, 2008.